

```
import numpy as np
a = np.array([[1, -2, 4],[2, 3, 12]])
b = np.array([[2,-1,3],[1,0,4]])
c = a+b
print(a)
print(b)
# para sumar las matrices deben tener el mismo número de filas y de columnas
print (c)
```

```
[[ 1 -2  4]
 [ 2  3 12]]
[[ 2 -1  3]
 [ 1  0  4]]
[[ 3 -3  7]
 [ 3  3 16]]
```

```
import numpy as np
a = np.array([[1, -2, 4],[2, 3, 12]])
b = np.array([[2,-1,3],[1,0,4]])
c = np.array([[1,4],[-5,3]])
#print(a+b)

escalar = 14
print(f"la matriz a es igual a {a}, y el producto por escalar es igual a {escalar*a}")
```

```
la matriz a es igual a [[ 1 -2  4]
 [ 2  3 12]], y el producto por escalar es igual a [[ 14 -28  56]
 [ 28  42 168]]
```

```
d = np.matrix([[3,1,0],[0,1,1]])
print(b+d)
```

```
e = np.matrix([[1,3,4],[5,1,1]])
f = np.matrix([[1,3],[1,4]])
g = np.matrix([[1,3],[1,4],[-1,5]])

print(f*e)
print(e*g)
```

```
[[16  6  7]
 [21  7  8]]
[[ 0 35]
 [ 5 24]]
```

```
#actividad: hacer cuatro martices con 3*2, 5*2, 3*2 y 2*2.
#hay que hacer una suma, resta, multiplicación por el escalar 3 y multiplicación entre matrices
import numpy as np

A1 = np.matrix([[1,2],[3,4],[1,3]])
A2 = np.matrix([[5,6],[7,8],[1,3],[2,5],[2,2]])
A3 = np.matrix([[9,10],[11,12],[7,1]])
A4 = np.matrix([[1,2],[5,5]])
escalar = 3

print(A1+A3)
print(A1-A3)
print(escalar*A2)
print(A1*A4)
```

```
[[10 12]
 [14 16]
 [ 8  4]]
[[-8 -8]
 [-8 -8]
 [-6  2]]
[[15 18]
 [21 24]
 [ 3  9]
 [ 6 15]
 [ 6  6]]
[[11 12]
```

```
[23 26]  
[16 17]]
```

Operaciones entre Matrices con NumPy en Python

Documento Técnico

10 de septiembre de 2026

Índice

1. Introducción	2
2. Conceptos Básicos	2
2.1. Creación de matrices	2
3. Operaciones Elementales	2
3.1. Suma y resta de matrices	2
3.2. Multiplicación por un escalar	3
4. Productos entre Matrices	3
4.1. Producto elemento a elemento (Hadamard)	3
4.2. Producto matricial	3
5. Operaciones Avanzadas	4
5.1. Transpuesta	4
5.2. Determinante	4
5.3. Inversa	4
5.4. Resolución de sistemas lineales	4
6. Operaciones Estadísticas y Reducciones	5
7. Otras Operaciones Útiles	5
8. Conclusión	6

1. Introducción

NumPy (Numerical Python) es una biblioteca fundamental para la computación científica en Python. Proporciona un objeto de arreglo multidimensional llamado `ndarray`, junto con una colección de rutinas para realizar operaciones rápidas sobre estos arreglos, incluyendo operaciones matriciales.

En este documento se describen las principales operaciones entre matrices utilizando NumPy, acompañadas de ejemplos prácticos.

2. Conceptos Básicos

2.1. Creación de matrices

Para crear matrices con NumPy, se utiliza la función `np.array()` pasando listas anidadas, o funciones auxiliares como `np.zeros()`, `np.ones()`, `np.eye()`, etc.

```
1 import numpy as np
2
3 # Matriz a partir de listas
4 A = np.array([[1, 2, 3],
5               [4, 5, 6]])
6
7 # Matriz identidad 3x3
8 I = np.eye(3)
9
10 # Matriz de ceros 2x3
11 Z = np.zeros((2, 3))
12
13 # Matriz de unos 3x2
14 O = np.ones((3, 2))
15
16 print("A =\n", A)
17 print("I =\n", I)
18 print("Z =\n", Z)
19 print("O =\n", O)
```

Listing 1: Creación de matrices básicas

3. Operaciones Elementales

3.1. Suma y resta de matrices

La suma y la resta se realizan elemento a elemento, y requieren que las matrices tengan las mismas dimensiones.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3 B = np.array([[5, 6],
4               [7, 8]])
5
6 # Suma
7 C = A + B          # o np.add(A, B)
8
9 # Resta
10 D = A - B          # o np.subtract(A, B)
11
12 print("A + B =\n", C)
13 print("A - B =\n", D)
```

Listing 2: Suma y resta de matrices

Resultado:

$$A + B = \begin{pmatrix} 6 & 8 \\ 10 & 12 \end{pmatrix}, \quad A - B = \begin{pmatrix} -4 & -4 \\ -4 & -4 \end{pmatrix}$$

3.2. Multiplicación por un escalar

Cada elemento de la matriz se multiplica por el escalar.

```
1 A = np.array([[1, 2], [3, 4]])
2 k = 3
3
4 C = k * A          # o np.multiply(k, A)
5
6 print("3 * A =\n", C)
```

Listing 3: Multiplicación por escalar

Resultado:

$$3A = \begin{pmatrix} 3 & 6 \\ 9 & 12 \end{pmatrix}$$

4. Productos entre Matrices

4.1. Producto elemento a elemento (Hadamard)

Se utiliza el operador `*` o la función `np.multiply()`. Las dimensiones deben coincidir.

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 C = A * B          # producto elemento a elemento
5
6 print("A * B =\n", C)
```

Listing 4: Producto elemento a elemento

Resultado:

$$A \odot B = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}$$

4.2. Producto matricial

El producto matricial se realiza con `np.dot()`, el operador `@`, o `np.matmul()`. El número de columnas de la primera matriz debe coincidir con el número de filas de la segunda.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3 B = np.array([[5, 6],
4               [7, 8]])
5
6 C = A @ B          # recomendado en Python >= 3.5
7 C2 = np.dot(A, B)  # equivalente
8 C3 = np.matmul(A, B) # equivalente
9
10 print("A @ B =\n", C)
```

Listing 5: Producto matricial

Resultado:

$$AB = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}$$

5. Operaciones Avanzadas

5.1. Transpuesta

Se obtiene con el atributo `.T` o la función `np.transpose()`.

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 At = A.T
5 print("A^T =\n", At)
```

Listing 6: Transpuesta de una matriz

Resultado:

$$A^T = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}$$

5.2. Determinante

Se calcula con `np.linalg.det()`. La matriz debe ser cuadrada.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3
4 det = np.linalg.det(A)
5 print("det(A) =", det)
```

Listing 7: Determinante

Resultado: $\det(A) = -2,0$

5.3. Inversa

Se calcula con `np.linalg.inv()`. La matriz debe ser cuadrada y no singular.

```
1 A = np.array([[1, 2],
2               [3, 4]])
3
4 A_inv = np.linalg.inv(A)
5 print("A^-1 =\n", A_inv)
6
7 # Verificaci n: A @ A^-1 = I
8 print("A @ A^-1 =\n", A @ A_inv)
```

Listing 8: Inversa de una matriz

Resultado:

$$A^{-1} = \begin{pmatrix} -2 & 1 \\ 1,5 & -0,5 \end{pmatrix}$$

5.4. Resolución de sistemas lineales

Para resolver $Ax = b$, se recomienda usar `np.linalg.solve()`, que es más eficiente y numéricamente estable que calcular la inversa.

```
1 A = np.array([[3, 1],
2               [1, 2]])
3 b = np.array([9, 8])
4
5 x = np.linalg.solve(A, b)
6 print("Soluci n x =", x)
```

Listing 9: Sistema de ecuaciones lineales

Resultado: $x = [2, 3]$

6. Operaciones Estadísticas y Reducciones

NumPy permite aplicar funciones estadísticas a matrices por filas o columnas usando el parámetro `axis`.

```
1 A = np.array([[1, 2, 3],
2               [4, 5, 6]])
3
4 # Suma total
5 total = np.sum(A)
6
7 # Suma por columnas (axis=0)
8 sum_cols = np.sum(A, axis=0)
9
10 # Suma por filas (axis=1)
11 sum_rows = np.sum(A, axis=1)
12
13 # Media
14 media = np.mean(A)
15
16 # Valor máximo
17 maximo = np.max(A)
18
19 print("Suma total:", total)
20 print("Suma por columnas:", sum_cols)
21 print("Suma por filas:", sum_rows)
22 print("Media:", media)
23 print("Máximo:", maximo)
```

Listing 10: Suma, media y otras reducciones

7. Otras Operaciones Útiles

- `np.concatenate([A, B], axis=0)`: concatena matrices por filas.
- `np.concatenate([A, B], axis=1)`: concatena matrices por columnas.
- `np.vstack([A, B])`: apila verticalmente.
- `np.hstack([A, B])`: apila horizontalmente.
- `np.linalg.matrix_rank(A)`: rango de la matriz.
- `np.linalg.eig(A)`: valores y vectores propios.
- `np.linalg.norm(A)`: norma matricial.

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 # Concatenación
5 V = np.vstack([A, B])
6 H = np.hstack([A, B])
7
8 # Rango
9 rango = np.linalg.matrix_rank(A)
10
11 # Valores y vectores propios
12 valores, vectores = np.linalg.eig(A)
13
14 print("Vstack =\n", V)
```

```
15 print("Hstack =\n", H)
16 print("Rango:", rango)
17 print("Valores propios:", valores)
```

Listing 11: Ejemplos adicionales

8. Conclusión

NumPy ofrece un conjunto amplio y eficiente de herramientas para trabajar con matrices en Python. Las operaciones descritas en este documento cubren la mayoría de los casos de uso típicos en álgebra lineal, análisis de datos y computación científica. Se recomienda consultar la documentación oficial de NumPy para profundizar en funciones más especializadas.

Referencias

- NumPy Documentation: <https://numpy.org/doc/>
- Harris, C. R., et al. (2020). *Array programming with NumPy*. Nature, 585, 357–362.

Operaciones entre Matrices con NumPy en Python

Documento de Referencia

10 de septiembre de 2026

Resumen

Este documento presenta una guía completa sobre las operaciones entre matrices utilizando la librería NumPy en Python. Se describen las operaciones fundamentales como suma, resta, multiplicación, división, transpuesta, inversa, determinante, producto punto, y operaciones elemento por elemento. Cada operación incluye un ejemplo práctico con código y su respectiva salida.

Índice

1. Introducción	3
2. Instalación e Importación	3
3. Creación de Matrices	3
4. Operaciones Aritméticas Básicas	3
4.1. Suma de Matrices	3
4.2. Resta de Matrices	4
4.3. Multiplicación por Escalar	4
5. Multiplicación de Matrices	4
5.1. Producto Punto (Multiplicación Matricial)	4
5.2. Multiplicación Elemento por Elemento (Hadamard)	5
6. División de Matrices	5
6.1. División Elemento por Elemento	5
6.2. División Escalar	5
7. Operaciones con Matrices Cuadradas	6
7.1. Transpuesta de una Matriz	6
7.2. Determinante de una Matriz	6
7.3. Inversa de una Matriz	6
8. Operaciones con Vectores	7
8.1. Producto Punto de Vectores	7
8.2. Norma de un Vector	7

9. Operaciones de Potencia	7
9.1. Potencia Elemento por Elemento	7
9.2. Potencia de Matriz (Matriz Elevada a una Potencia)	7
10.Otras Operaciones Útiles	8
10.1. Suma de Elementos	8
10.2. Media de Elementos	8
11.Ejemplo Completo: Sistema de Ecuaciones Lineales	9
12.Conclusión	9

1. Introducción

NumPy es una librería fundamental para el cálculo científico en Python. Proporciona objetos multidimensionales (arrays) y herramientas para manipularlos eficientemente. Las operaciones con matrices son esenciales en álgebra lineal, machine learning, procesamiento de imágenes y muchas otras áreas.

2. Instalación e Importación

Para utilizar NumPy, primero debemos instalarlo y luego importarlo en nuestro script:

```
1 # Instalación (desde terminal)
2 # pip install numpy
3
4 # Importación
5 import numpy as np
```

3. Creación de Matrices

Antes de realizar operaciones, necesitamos crear matrices. NumPy ofrece varias formas de hacerlo:

```
1 # Creación de matrices
2 A = np.array([[1, 2, 3],
3              [4, 5, 6],
4              [7, 8, 9]])
5
6 B = np.array([[9, 8, 7],
7              [6, 5, 4],
8              [3, 2, 1]])
9
10 # Matriz identidad
11 I = np.eye(3)
12
13 # Matriz de ceros
14 Z = np.zeros((3, 3))
15
16 # Matriz de unos
17 O = np.ones((3, 3))
18
19 # Matriz aleatoria
20 R = np.random.random((3, 3))
21
22 print("Matriz A:\n", A)
23 print("Matriz B:\n", B)
```

4. Operaciones Aritméticas Básicas

4.1. Suma de Matrices

La suma de matrices se realiza elemento por elemento. Las matrices deben tener las mismas dimensiones.

```

1 # Suma de matrices
2 A = np.array([[1, 2], [3, 4]])
3 B = np.array([[5, 6], [7, 8]])
4 C = A + B
5
6 print("A:\n", A)
7 print("B:\n", B)
8 print("A + B:\n", C)
9
10 # Salida:
11 # A + B = [[ 6  8]
12 #          [10 12]]

```

4.2. Resta de Matrices

La resta también se realiza elemento por elemento.

```

1 # Resta de matrices
2 C = A - B
3
4 print("A - B:\n", C)
5
6 # Salida:
7 # A - B = [[-4 -4]
8 #          [-4 -4]]

```

4.3. Multiplicación por Escalar

Multiplicar una matriz por un escalar multiplica cada elemento por ese valor.

```

1 # Multiplicación por escalar
2 k = 3
3 C = k * A
4
5 print("3 * A:\n", C)
6
7 # Salida:
8 # 3 * A = [[ 3  6]
9 #          [ 9 12]]

```

5. Multiplicación de Matrices

5.1. Producto Punto (Multiplicación Matricial)

La multiplicación de matrices (producto punto) se realiza con la función `np.dot()` o el operador `@`.

```

1 # Producto punto (multiplicación matricial)
2 A = np.array([[1, 2], [3, 4]])
3 B = np.array([[5, 6], [7, 8]])
4
5 # M todo 1: usando np.dot()
6 C = np.dot(A, B)

```

```

7
8 # M todo 2: usando el operador @
9 D = A @ B
10
11 print("A @ B:\n", C)
12 print("A @ B (operador @):\n", D)
13
14 # Salida:
15 # A @ B = [[19 22]
16 #          [43 50]]

```

5.2. Multiplicación Elemento por Elemento (Hadamard)

La multiplicación elemento por elemento (producto de Hadamard) se realiza con el operador `*`.

```

1 # Multiplicaci n elemento por elemento (Hadamard)
2 C = A * B
3
4 print("A * B (elemento por elemento):\n", C)
5
6 # Salida:
7 # A * B = [[ 5 12]
8 #          [21 32]]

```

6. División de Matrices

6.1. División Elemento por Elemento

La división se realiza elemento por elemento con el operador `/`.

```

1 # Divisi n elemento por elemento
2 A = np.array([[10, 20], [30, 40]])
3 B = np.array([[2, 4], [5, 8]])
4 C = A / B
5
6 print("A / B (elemento por elemento):\n", C)
7
8 # Salida:
9 # A / B = [[5. 5.]
10 #          [6. 5.]]

```

6.2. División Escalar

También podemos dividir una matriz por un escalar.

```

1 # Divisi n por escalar
2 C = A / 2
3
4 print("A / 2:\n", C)
5
6 # Salida:
7 # A / 2 = [[ 5. 10.]
8 #          [15. 20.]]

```

7. Operaciones con Matrices Cuadradas

7.1. Transpuesta de una Matriz

La transpuesta se obtiene con `np.transpose()` o `.T`.

```
1 # Transpuesta de una matriz
2 A = np.array([[1, 2, 3],
3               [4, 5, 6],
4               [7, 8, 9]])
5
6 A_T = np.transpose(A) # o A.T
7
8 print("Matriz A:\n", A)
9 print("Transpuesta de A:\n", A_T)
10
11 # Salida:
12 # Transpuesta de A:
13 # [[1 4 7]
14 #  [2 5 8]
15 #  [3 6 9]]
```

7.2. Determinante de una Matriz

El determinante se calcula con `np.linalg.det()`.

```
1 # Determinante de una matriz
2 A = np.array([[1, 2], [3, 4]])
3 det_A = np.linalg.det(A)
4
5 print("Matriz A:\n", A)
6 print(f"Determinante de A: {det_A:.2f}")
7
8 # Salida:
9 # Determinante de A: -2.00
```

7.3. Inversa de una Matriz

La inversa se calcula con `np.linalg.inv()`.

```
1 # Inversa de una matriz
2 A = np.array([[4, 7], [2, 6]])
3 A_inv = np.linalg.inv(A)
4
5 print("Matriz A:\n", A)
6 print("Inversa de A:\n", A_inv)
7
8 # Verificación: A * A_inv = I
9 print("A * A_inv:\n", A @ A_inv)
10
11 # Salida:
12 # Inversa de A:
13 # [[ 0.6 -0.7]
14 # [-0.2  0.4]]
```

8. Operaciones con Vectores

8.1. Producto Punto de Vectores

El producto punto de vectores se calcula con `np.dot()`.

```
1 # Producto punto de vectores
2 v = np.array([1, 2, 3])
3 w = np.array([4, 5, 6])
4
5 producto_punto = np.dot(v, w) # o v @ w
6
7 print(f"v = {v}")
8 print(f"w = {w}")
9 print(f"v      w = {producto_punto}")
10
11 # Salida:
12 # v      w = 32  (1*4 + 2*5 + 3*6)
```

8.2. Norma de un Vector

La norma se calcula con `np.linalg.norm()`.

```
1 # Norma de un vector
2 v = np.array([3, 4])
3 norma = np.linalg.norm(v)
4
5 print(f"v = {v}")
6 print(f"Norma de v: {norma:.2f}")
7
8 # Salida:
9 # Norma de v: 5.00  ( (3  + 4  ) )
```

9. Operaciones de Potencia

9.1. Potencia Elemento por Elemento

Elevar cada elemento de la matriz a una potencia.

```
1 # Potencia elemento por elemento
2 A = np.array([[1, 2], [3, 4]])
3 C = A ** 2
4
5 print("A:\n", A)
6 print("A ** 2:\n", C)
7
8 # Salida:
9 # A ** 2 = [[ 1  4]
10 #           [ 9 16]]
```

9.2. Potencia de Matriz (Matriz Elevada a una Potencia)

Elevar una matriz cuadrada a una potencia usando multiplicación matricial.

```

1 # Potencia de matriz (solo para matrices cuadradas)
2 A = np.array([[1, 2], [3, 4]])
3 A_potencia = np.linalg.matrix_power(A, 2) # A^2
4
5 print("A:\n", A)
6 print("A^2:\n", A_potencia)
7
8 # Salida:
9 # A^2 = [[ 7 10]
10 #        [15 22]]

```

10. Otras Operaciones Útiles

10.1. Suma de Elementos

Sumar todos los elementos de una matriz.

```

1 # Suma de todos los elementos
2 A = np.array([[1, 2, 3], [4, 5, 6]])
3 suma_total = np.sum(A)
4 suma_filas = np.sum(A, axis=1) # Suma por filas
5 suma_columnas = np.sum(A, axis=0) # Suma por columnas
6
7 print("A:\n", A)
8 print(f"Suma total: {suma_total}")
9 print(f"Suma por filas: {suma_filas}")
10 print(f"Suma por columnas: {suma_columnas}")
11
12 # Salida:
13 # Suma total: 21
14 # Suma por filas: [ 6 15]
15 # Suma por columnas: [5 7 9]

```

10.2. Media de Elementos

Calcular la media de los elementos.

```

1 # Media de los elementos
2 media_total = np.mean(A)
3 media_filas = np.mean(A, axis=1)
4 media_columnas = np.mean(A, axis=0)
5
6 print(f"Media total: {media_total:.2f}")
7 print(f"Media por filas: {media_filas}")
8 print(f"Media por columnas: {media_columnas}")
9
10 # Salida:
11 # Media total: 3.50
12 # Media por filas: [2. 5.]
13 # Media por columnas: [2.5 3.5 4.5]

```

11. Ejemplo Completo: Sistema de Ecuaciones Lineales

Resolvamos un sistema de ecuaciones lineales usando NumPy:

```
1 # Resolver sistema de ecuaciones:
2 # 2x + 3y = 8
3 # 4x + y = 6
4
5 # Matriz de coeficientes
6 A = np.array([[2, 3], [4, 1]])
7 # Vector de constantes
8 b = np.array([8, 6])
9
10 # Resolver el sistema: A * x = b
11 x = np.linalg.solve(A, b)
12
13 print("Matriz de coeficientes A:\n", A)
14 print("Vector de constantes b:", b)
15 print("Soluci n del sistema:", x)
16
17 # Verificaci n: A * x debe ser igual a b
18 verificacion = A @ x
19 print("Verificaci n A * x:", verificacion)
20
21 # Salida:
22 # Soluci n del sistema: [1. 2.]
23 # Verificaci n A * x: [8. 6.]
```

12. Conclusión

NumPy proporciona una amplia gama de operaciones para trabajar con matrices de manera eficiente en Python. Las operaciones descritas en este documento son fundamentales para el álgebra lineal y tienen aplicaciones en:

- Machine Learning y Deep Learning
- Procesamiento de imágenes
- Simulaciones científicas
- Análisis de datos
- Optimización numérica

La combinación de la sintaxis intuitiva de Python con el rendimiento optimizado de NumPy hace que estas operaciones sean tanto accesibles como potentes.

Referencias

1. NumPy Documentation: <https://numpy.org/doc/>
2. Harris, C.R., et al. (2020). "Array programming with NumPy". Nature, 585, 357–362.

3. Oliphant, T.E. (2006). "Guide to NumPy". Continuum Press.

```
import numpy as np
arreglo = np.array([10,20,30,40,50])
suma = 0

for valor in arreglo:
    suma += valor
print(f"la suma del arreglo es:{suma}")
```

la suma del arreglo es:150

```
matriz = [
    [1,2,3],
    [4,5,6],
    [7,8,9]
]

for fila in matriz:
    for elemento in fila:
        print(elemento, end=" ")
    print() #salto de linea al terminar cada fila
```

1 2 3
4 5 6
7 8 9

```
for fila in matriz:
    print(fila)
```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

```
for fila in matriz:
    for elemento in fila:
        print(elemento)
```

1
2
3
4
5
6
7
8
9

```
ms = [
    [3,5,7],
    [-1,2,7],
]
print (ms)
```

[[3, 5, 7], [-1, 2, 7]]

```
for fila in ms:
    for elemento in fila:
        print(elemento, end=" ")
    print()
```

3 5 7
-1 2 7

```
print(ms [0][1])
```

5

```
def andrea_det2x2(A):
    a = A[0][0]
    b = A[0][1]
    c = A[1][0]
```

```
d = A[1][1]
y = a*d-b*c
return y
```

```
A3 = [
    [1, 3],
    [4, 5],
]
print(A3)
andrea_det2x2(A3)
```

```
[[1, 3], [4, 5]]
-7
```

```
def andrea_det3x3(B):
    a = B[0][0]
    b = B[0][1]
    c = B[0][2]
    d = B[1][0]
    e = B[1][1]
    f = B[1][2]
    g = B[2][0]
    h = B[2][1]
    i = B[2][2]
    B1 = [[e,f],[h,i]]
    B2 = [[d,f],[g,i]]
    B3 = [[d,e],[g,h]]
    y = a*andrea_det2x2(B1)-b*andrea_det2x2(B2)+c*andrea_det2x2(B3)
    return y
```

```
Z = [
    [1,2,7],
    [4,3,4],
    [5,6,7]
]
print(Z)
andrea_det3x3(Z)
```

```
[[1, 2, 7], [4, 3, 4], [5, 6, 7]]
44
```

```
X = andrea_det2x2([[4,2],[8,5]])/andrea_det2x2([[3,2],[-1,5]])
print(X)
```

```
0.23529411764705882
```

```
y = andrea_det2x2([[3,4],[-1,8]])/andrea_det2x2([[3,2],[-1,5]])
print(f"la solucion es:{y:.3f}")
```

```
la solucion es:1.647
```

Start coding or [generate](#) with AI.

